

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP.HỒ CHÍ MINH

KHOA ĐIỆN – ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH – VIỄN THÔNG



HCMUTE

ĐỒ ÁN TỐT NGHIỆP

THIẾT KẾ BỘ VI XỬ LÝ 32 BIT

NGÀNH : CNKT ĐIỆN TỬ - VIỄN THÔNG

SVTH : NGUYỄN BẢO TÍNH

MSSV: 20161383

TRẦN NHẬT HẢI

MSSV: 20161311

Tp. Hồ Chí Minh, 06/2024

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP.HỒ CHÍ MINH

KHOA ĐIỆN – ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH – VIỄN THÔNG



HCMUTE

ĐỒ ÁN TỐT NGHIỆP

THIẾT KẾ BỘ VI XỬ LÝ 32 BIT

NGÀNH : CNKT ĐIỆN TỬ - VIỄN THÔNG

SVTH: NGUYỄN BẢO TÍNH

MSSV: 20161383

TRẦN NHẬT HẢI

MSSV: 20161311

GVHD : ThS. LÊ MINH THÀNH

Tp. Hồ Chí Minh, 06/2024

MỤC LỤC

CHƯƠNG 1: TỔNG QUAN.....	1
1.1. TỔNG QUAN.....	1
1.2. MỤC TIÊU ĐỀ TÀI.....	2
1.3. GIỚI HẠN ĐỀ TÀI.....	3
1.4. BỐ CỤC ĐỒ ÁN.....	3
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT (Được trình bày chi tiết trong cuốn báo cáo đầy đủ).....	3
CHƯƠNG 3: THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG.....	4
3.1. YÊU CẦU HỆ THỐNG	4
3.2. THIẾT KẾ SƠ ĐỒ KHỐI VÀ CHỨC NĂNG MỖI KHỐI.....	4
3.2.1. Sơ đồ khối hệ thống.....	4
3.2.2. Hoạt động hệ thống	4
3.2.3. Khối lấy lệnh	6
3.2.4. Khối giải mã lệnh	6
3.2.5. Khối thực thi.....	7
3.2.6. Khối truy cập bộ nhớ.....	8
3.2.7. Khối ghi lại.....	8
3.2.8. Khối xử lý xung đột.....	9
CHƯƠNG 4: KẾT QUẢ.....	10
4.1. KẾT QUẢ MÔ PHỎNG DẠNG SÓNG CÁC KHỐI.....	10
4.1.1. Khối lấy lệnh	10
4.1.2. Khối giải mã lệnh	11
4.1.3. Khối thực thi.....	13
4.1.4. Khối truy cập bộ nhớ.....	15
4.1.5. Khối ghi lại.....	17
4.1.6. Khối xử lý xung đột.....	18
4.2. KẾT QUẢ MÔ PHỎNG TOÀN HỆ THỐNG	19

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	21
5.1. KẾT LUẬN.....	26
5.2. HƯỚNG PHÁT TRIỂN	26
TÀI LIỆU THAM KHẢO.....	27

CHƯƠNG 1: TỔNG QUAN

1.1. TỔNG QUAN

Thế giới hiện đại đang trải qua sự phát triển công nghệ mạnh mẽ, đem lại những đổi mới kỹ thuật số quan trọng, ảnh hưởng đến các ngành công nghiệp và cấu trúc xã hội, thay đổi nhiều khía cạnh của cuộc sống. Cuộc cách mạng công nghiệp 4.0 đặc biệt bởi sự kết hợp và tương tác giữa nhiều xu hướng công nghệ, tạo nên sức mạnh mới và đột phá [1].

Ngành công nghệ bán dẫn đã trở thành ngành công nghiệp quan trọng, đặc biệt trong bối cảnh thiếu hụt chip toàn cầu. Ông Phil Hoàng từ Skyworks Solutions cho biết vi mạch bán dẫn là nền tảng của tính toán hiện đại, quan trọng trong giai đoạn bùng nổ nghiên cứu và sản xuất chip. Ngành công nghiệp này đóng vai trò thiết yếu trong chuyển đổi số, phát triển kinh tế tri thức, đặc biệt tại Việt Nam với nguồn nhân lực chăm chỉ và đội ngũ khoa học năng động. Công nghệ RISC, với ưu điểm giảm lệnh máy, tối ưu hiệu suất, tiêu thụ năng lượng thấp, đang ngày càng phổ biến trong thiết kế vi mạch và máy tính, ứng dụng rộng rãi trong các thiết bị điện tử và IoT.

Ở đề tài “Design of 32-bit Processor for Embedded Systems” của Hyun Woo Oh, Kwon Neung Cho, và Seung Eun Lee đã áp dụng kiến trúc MIPS cho bộ vi xử lý 32-bit cho hệ thống nhúng, đạt được ưu điểm diện tích nhỏ và công suất tiêu thụ thấp, nhưng còn hạn chế ở khả năng xử lý đa nhiệm so với các kiến trúc RISC hiện đại. Đề tài “A Single Clock Cycle MIPS RISC Processor Design using VHDL” của Mamun Bin Ibne Reaz, Md. Shabiul Islam và Mohd. S. Sulaiman, và “Hardware Modelling of a 32-bit Single Cycle RISC Processor using VHDL” của Safaa S. Omran và Hadeel S. Mahmood sử dụng một chu kỳ xung duy nhất cho vi xử lý MIPS RISC, giúp thiết kế đơn giản và không có xung đột dữ liệu. Tuy nhiên, hiệu suất bị giảm do thời gian thực thi trung bình của mỗi lệnh tăng và lãng phí tài nguyên.

Đề án này tập trung phát triển một bộ vi xử lý mã nguồn mở dựa trên công nghệ RISC, có khả năng mở rộng, tiết kiệm năng lượng và hiệu suất cao. RISC-V tuân

thủ nguyên tắc kiến trúc tập lệnh đơn giản và hoạt động như một tập lệnh mã nguồn mở, miễn phí cho cá nhân và doanh nghiệp. RISC-V được phát triển bởi nhóm dẫn đầu bởi Giáo sư David Patterson tại Đại học California, Berkeley, nhằm đáp ứng nhu cầu về một kiến trúc tập lệnh mở, linh hoạt, có thể mở rộng và hỗ trợ các không gian địa chỉ 32 bit, 64 bit và 128 bit .

Đồ án này phát triển bộ vi xử lý dựa trên kiến trúc RISC-V và kiến trúc Harvard, nhằm dễ dàng triển khai với kiến trúc linh hoạt, tiêu chuẩn. Nhóm sẽ thiết kế các thành phần chính: bộ điều khiển, bộ nhớ, bộ xử lý lệnh, và bộ xử lý dữ liệu để tối ưu hiệu suất. Phần mềm mô phỏng Vivado Design Suite sẽ được dùng để kiểm tra tính đúng đắn và hiệu suất của bộ vi xử lý. Nhóm cải tiến từ các nghiên cứu trước bằng cách sử dụng kiến trúc Harvard với hai bộ nhớ riêng biệt cho lệnh và dữ liệu, và kỹ thuật đường ống cho phép nhiều lệnh thực hiện đồng thời, tăng tốc độ xử lý và tối ưu hiệu suất. Kết quả dự kiến là bộ vi xử lý 32 bit dựa trên RISC-V và kiến trúc Harvard, có khả năng thực thi nhiều lệnh cùng lúc, tối ưu hiệu suất cho các hệ thống nhúng.

1.2. MỤC TIÊU ĐỀ TÀI

Mục tiêu của đề tài là phát triển một bộ vi xử lý 32 bit dựa trên kiến trúc tập lệnh RISC-V và kiến trúc Harvard, nhằm tối ưu hóa hiệu suất và tính hiệu quả trong các hệ thống nhúng :

- Tận dụng kiến trúc với tập lệnh đơn giản hóa để giảm độ phức tạp và mã nguồn mở hoàn toàn có thể truy cập linh hoạt để bất kỳ cá nhân nào sử dụng mà không chịu bất kỳ chi phí nào.
- Tận dụng kỹ thuật đường ống để tăng cường hiệu suất xử lý thông qua việc thực thi đồng thời các giai đoạn khác nhau của một lệnh trong mỗi chu kỳ xung.
- Tận dụng ưu điểm của kiến trúc Harvard để phân biệt bộ nhớ lệnh và bộ nhớ dữ liệu để cải thiện việc truy cập bộ nhớ, tăng tốc độ thực thi chương trình và tránh xung đột giữa việc đọc và ghi dữ liệu.

1.3. GIỚI HẠN ĐỀ TÀI

Đề tài sẽ tập trung vào việc thiết kế và phát triển một bộ vi xử lý 32 bit dựa trên kiến trúc tập lệnh RISC-V và kiến trúc Harvard. Tuy nhiên, để đảm bảo tính khả thi và hiệu quả, đề tài sẽ giới hạn vào các khía cạnh như:

- Hạn chế đi sâu vào vấn đề thiết kế vi mạch hay tích hợp ở cấp độ chip. Thay vào đó là thiết kế các thành phần cơ bản của bộ vi xử lý.
- Sử dụng phần mềm mô phỏng để kiểm tra tính đúng đắn và hiệu suất của bộ vi xử lý, nhưng không đặt mục tiêu vào việc tạo ra một mô hình mô phỏng hoàn chỉnh và chi tiết của toàn bộ bộ vi xử lý.
- Không tích hợp bộ vi xử lý với các công nghệ, giao thức khác trong hệ thống nhúng và nhấn mạnh vào việc phát triển một vi xử lý độc lập.

1.4. BỐ CỤC ĐỒ ÁN

Đề tài gồm 5 chương được phân bổ như sau:

- Chương 1: Tổng quan: Tổng quan về đề tài, mục tiêu đặt ra của đề tài, giới hạn đề tài cũng như bố cục của đề tài.
- Chương 2: Cơ sở lý thuyết: Lý thuyết liên quan đến đề tài, những khái niệm và nội dung cần tìm hiểu khi xây dựng đề tài như vi xử lý, bộ nhớ, kiến trúc máy tính...
- Chương 3: Thiết kế và xây dựng hệ thống: Mô tả yêu cầu của hệ thống, sơ đồ khối hệ thống, thiết kế chi tiết từng khối và thiết kế phần mềm.
- Chương 4: Kết quả: Đánh giá chức năng từng khối, kết quả mô phỏng và đánh giá khả năng đọc/ghi bộ nhớ.
- Chương 5: Kết luận và hướng phát triển.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT (Được trình bày chi tiết trong cuốn báo cáo đầy đủ)

CHƯƠNG 3: THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG

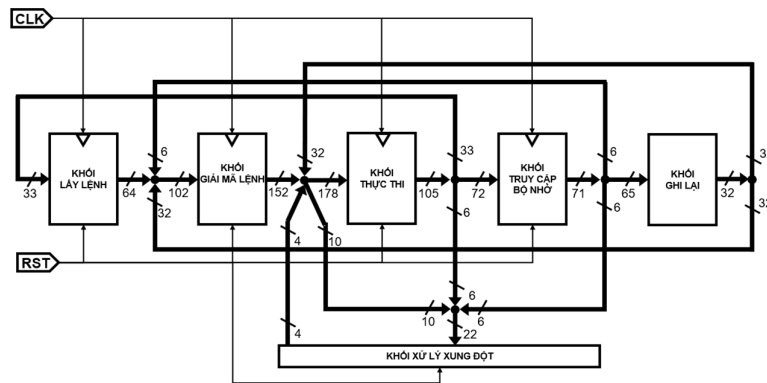
3.1. YÊU CẦU HỆ THỐNG

- Thiết kế dựa trên kiến trúc RISC-V.
- Hệ thống sử dụng kỹ thuật đường ống 5 giai đoạn.
- Kiến trúc Harvard.
- Thiết kế bằng Verilog HDL.
- Kiểm tra và xác minh.

3.2. THIẾT KẾ SƠ ĐỒ KHỐI VÀ CHỨC NĂNG MỖI KHỐI

3.2.1. Sơ đồ khối hệ thống

Hình 3.1 minh họa cấu trúc tổng quan của hệ thống vi xử lý 32 bit sử dụng kiến trúc RISC-V và kiến trúc Harvard với kỹ thuật đường ống 5 giai đoạn. Cấu trúc này bao gồm các khối chức năng: lấy lệnh, giải mã, thực thi, truy cập bộ nhớ, ghi lại và xử lý xung đột.

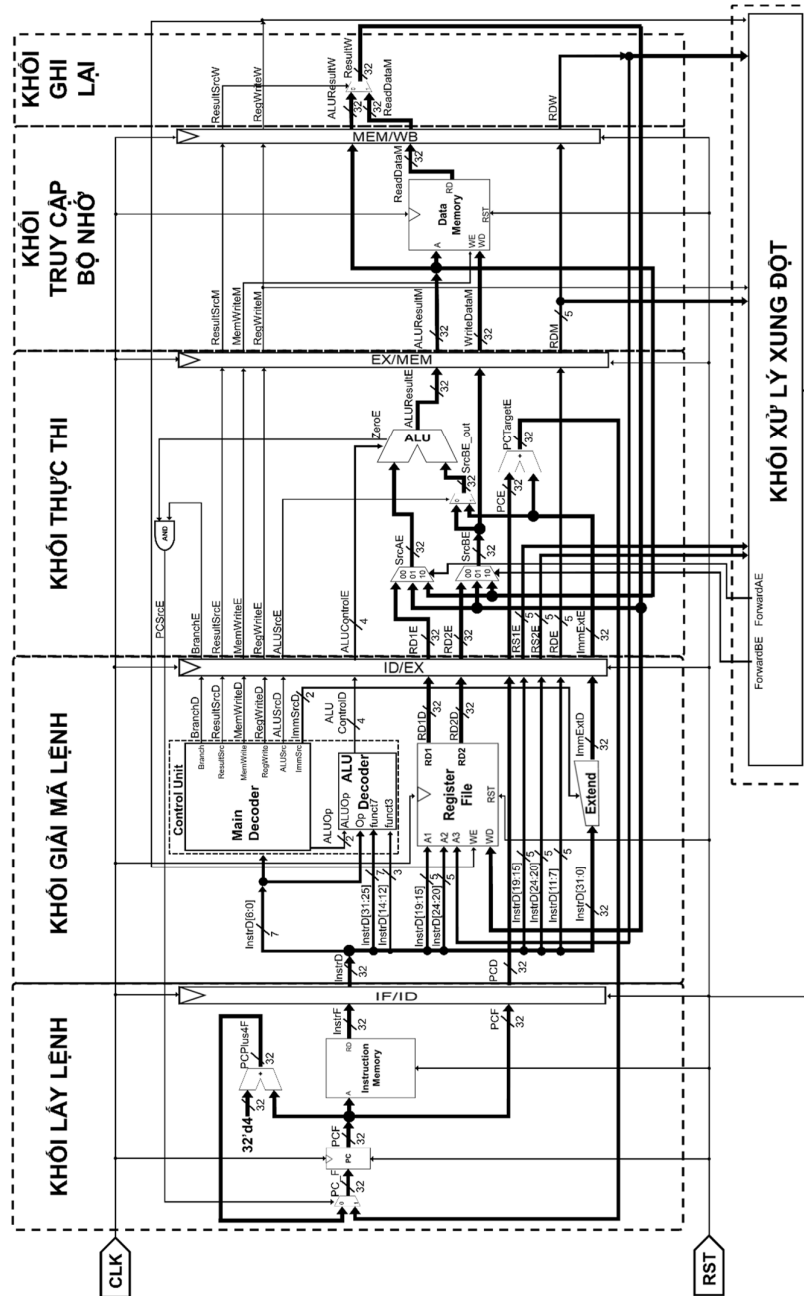


Hình 3.1: Sơ đồ tổng quan hệ thống

3.2.2. Hoạt động hệ thống

Hình 3.2 mô tả hệ thống vi xử lý 32 bit RISC-V với kỹ thuật đường ống 5 tầng để tối ưu hiệu suất. Các giai đoạn bao gồm: lấy lệnh từ địa chỉ PC và cập nhật PC, giải mã lệnh để xác định thanh ghi nguồn, đích, loại thao tác và tạo tín hiệu điều khiển, thực thi lệnh với ALU thực hiện tính toán hoặc logic và lưu kết quả vào thanh ghi EX/MEM, truy cập bộ nhớ để đọc/ghi dữ liệu và lưu kết quả vào thanh

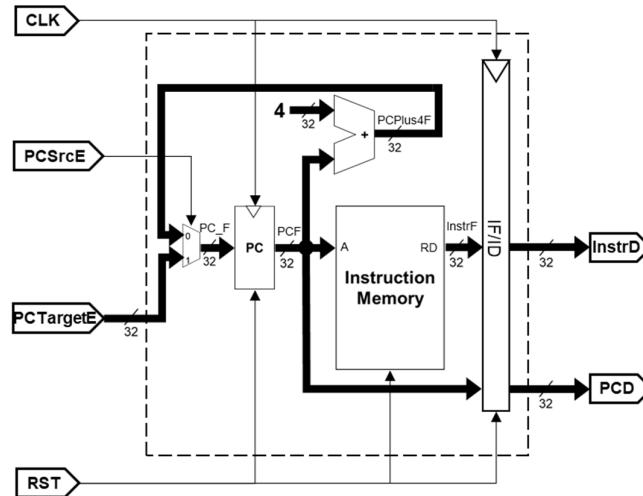
ghi MEM/WB, và cuối cùng là ghi lại để cập nhật thanh ghi đích với kết quả cuối cùng. Hệ thống sử dụng các khối thành ghi trung gian và đơn vị điều khiển để quản lý dòng chảy dữ liệu và tín hiệu, xử lý xung đột hiệu quả, cho phép thực thi nhiều lệnh cùng lúc và tăng hiệu suất tổng thể.



Hình 3.2: Sơ đồ chi tiết hệ thống

3.2.3. Khởi lấy lệnh

Sơ đồ chi tiết gồm các mô đun: bộ ghép kênh chọn đầu vào của chương trình, bộ cộng, bộ đếm chương trình, bộ nhớ lệnh và thanh ghi IF/ID.



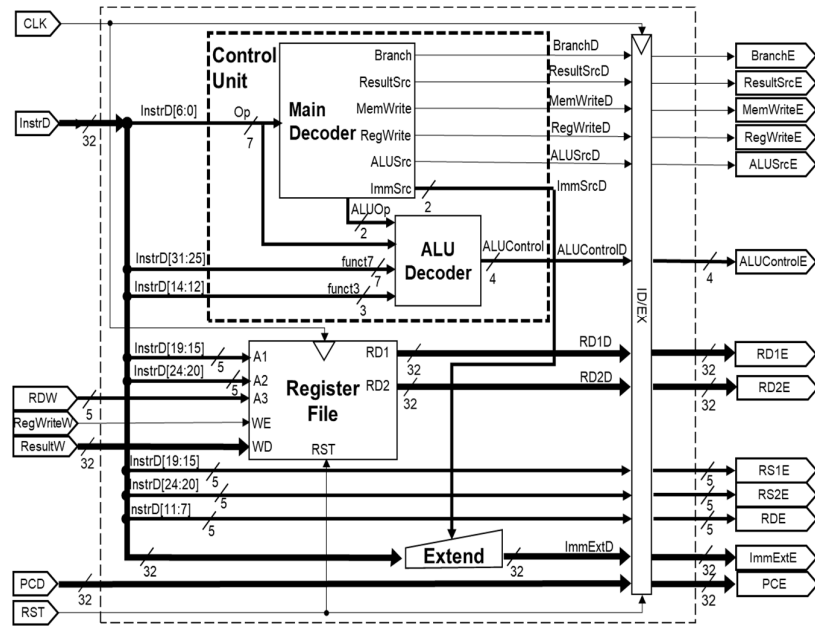
Hình 3.3: Sơ đồ chi tiết Khởi lấy lệnh

Khởi lấy lệnh trong Hình 3.3 thực hiện nhiệm vụ lấy lệnh từ bộ nhớ lệnh và cập nhật bộ đếm chương trình (PC). Quá trình này bao gồm các bước sau: tín hiệu điều khiển PCSrcE quyết định nguồn giá trị mới của PC, với địa chỉ đích là PCTargetE khi PCSrcE = 1 và PCplus4F là PC cộng thêm 4 để trở đến lệnh kế tiếp. Bộ ghép kênh chọn đầu vào cho PC, chọn PC + 4 khi PCSrcE = 0 và chọn PCTargetE khi PCSrcE = 1. Bộ đếm chương trình lưu và cập nhật giá trị hiện tại của PC mỗi chu kỳ xung đồng hồ. Bộ nhớ lệnh lưu trữ lệnh chương trình, với địa chỉ A là giá trị của PC và đầu ra RD (InstrF) là lệnh tại địa chỉ đó. Thanh ghi IF/ID lưu lệnh (InstrF) và giá trị PC (PCF) để chuyển sang giai đoạn giải mã lệnh (InstrD và PCD).

3.2.4. Khởi giải mã lệnh

Sơ đồ chi tiết gồm các mô đun: Đơn vị điều khiển có bộ giải mã chính và bộ giải mã ALU, tệp thanh ghi, khối mở rộng và thanh ghi ID/EX.

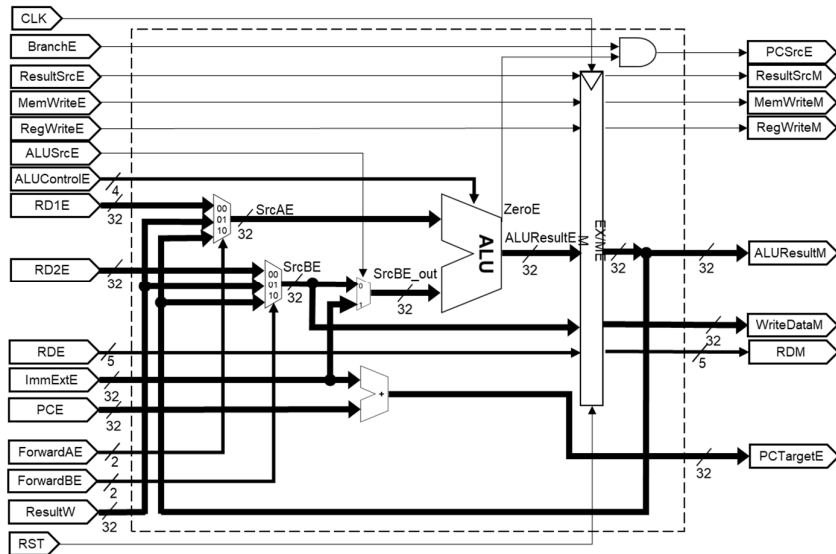
Khởi giải mã lệnh ở Hình 3.4 có nhiệm vụ phân tích lệnh và tạo ra các tín hiệu điều khiển cần thiết để thực hiện lệnh. Quá trình giải mã bao gồm việc xác định loại lệnh, các tham số của lệnh, và điều khiển các đơn vị chức năng khác trong bộ vi xử lý 32 bit.



Hình 3.4: Sơ đồ chi tiết Khối giải mã lệnh

3.2.5. Khối thực thi

Sơ đồ chi tiết gồm: Đơn vị luận lý số học, bộ cộng, bộ ghép kênh, thanh ghi EX/MEM.



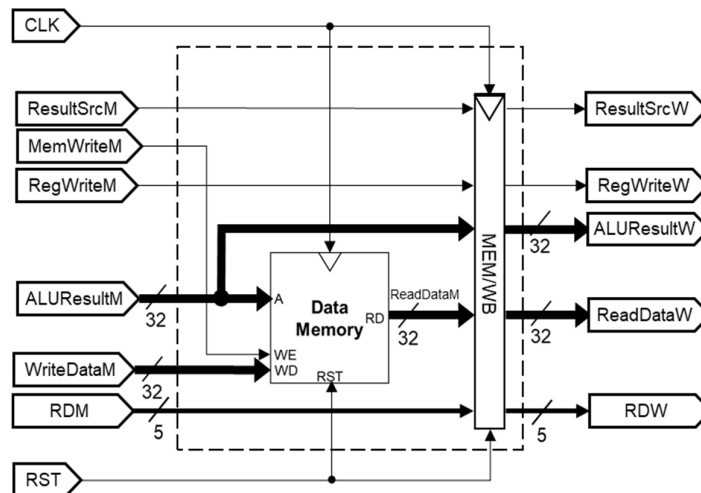
Hình 3.5: Sơ đồ chi tiết Khối thực thi

Khối thực thi ở Hình 3.5 thực hiện các phép toán và hoạt động dựa trên lệnh giải mã từ khối giải mã lệnh. ALU thực hiện các phép toán số học và logic theo tín hiệu ALUControlE, sử dụng giá trị từ thanh ghi, giá trị tức thời hoặc giá trị chuyển

tiếp, với tín hiệu ZeroE chỉ ra khi kết quả bằng không để quyết định nhánh. Xử lý xung đột được giải quyết bằng cách chuyển tiếp dữ liệu từ các giai đoạn sau về giai đoạn thực thi thông qua tín hiệu ForwardAE và ForwardBE. Bộ chọn quyết định sử dụng dữ liệu chuyển tiếp, dữ liệu thanh ghi hoặc giá trị tức thời cho ALU. Các tín hiệu điều khiển BranchE, ResultSrcE, MemWriteE, RegWriteE, ALUSrcE, ALUControlE và ResultW điều khiển hành động trong khối thực thi. Kết quả đầu ra ALUResultM chuyển đến giai đoạn bộ nhớ và WriteDataM ghi dữ liệu vào bộ nhớ, trong khi PCTargetE là địa chỉ mục tiêu cho lệnh nhánh.

3.2.6. Khối truy cập bộ nhớ

Sơ đồ chi tiết gồm bộ nhớ dữ liệu và thanh ghi MEM/WB



Hình 3.6: Sơ đồ chi tiết Khối truy cập bộ nhớ

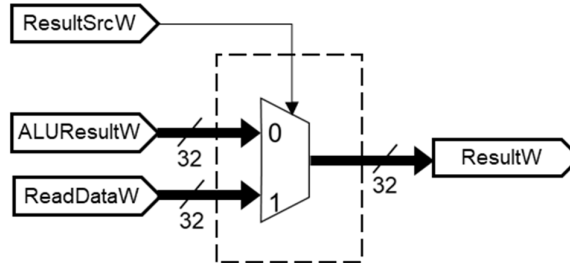
Hình 3.6 mô tả sơ đồ chi tiết của khối truy cập bộ nhớ, thực hiện chức năng đọc và ghi dữ liệu vào bộ nhớ chính. Khối này bao gồm tín hiệu MemWriteM điều khiển việc ghi dữ liệu, tín hiệu ALUResultM cung cấp địa chỉ từ ALU để ghi vào bộ nhớ, và tín hiệu WriteDataM chứa dữ liệu cần ghi. Thanh ghi MEM/WB lưu trữ các tín hiệu điều khiển và giá trị cần thiết để chuyển sang giai đoạn thực thi lệnh tiếp theo.

3.2.7. Khối ghi lại

Sơ đồ chi tiết chỉ có một bộ ghép kênh 2 ngõ vào.

Hình 3.7 mô tả khối ghi lại, có chức năng chọn kết quả cuối cùng để ghi vào

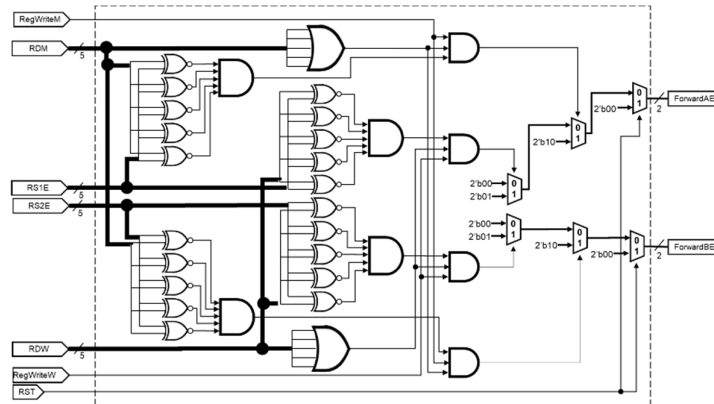
các thanh ghi. Tín hiệu ResultSrcW điều khiển việc chọn nguồn dữ liệu kết quả, gồm ALUResultW từ ALU và ReadDataW từ bộ nhớ. Bộ chọn quyết định dữ liệu nào sẽ được truyền ra ngoài dựa trên tín hiệu ResultSrcW.



Hình 3.7: Sơ đồ chi tiết Khối ghi lại

3.2.8. Khối xử lý xung đột

Khối xử lý xung đột trong sơ đồ này có nhiệm vụ phát hiện và giải quyết các xung đột xảy ra trong quá trình thực hiện lệnh.



Hình 3.8: Sơ đồ chi tiết Khối xử lý xung đột

Hình 3.8 mô tả về sơ đồ Khối xử lý xung đột có hai ngõ ra ForwardAE, ForwardBE được sử dụng để chọn nguồn dữ liệu phù hợp để tránh xung đột:

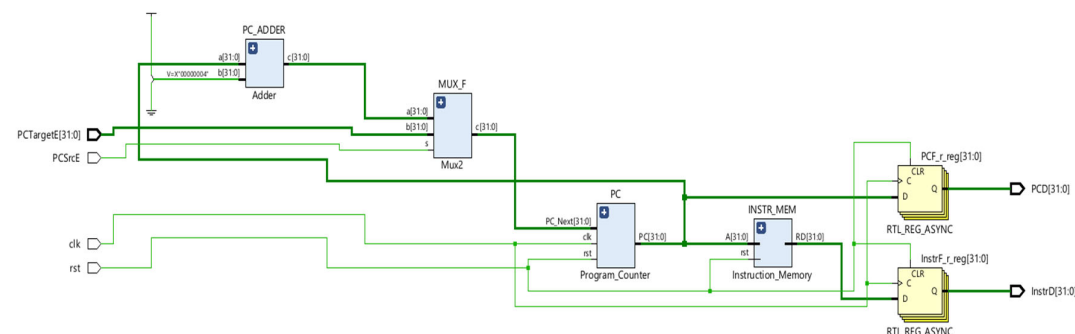
- Nếu có xung đột, khối sẽ điều khiển để chuyển tiếp dữ liệu từ giai đoạn trước đến giai đoạn hiện tại, đảm bảo rằng lệnh hiện tại có được dữ liệu chính xác mà không cần chờ đợi.
- Nếu không có xung đột, các tín hiệu này sẽ cho phép dữ liệu được sử dụng trực tiếp từ các thanh ghi nguồn.

CHƯƠNG 4: KẾT QUẢ

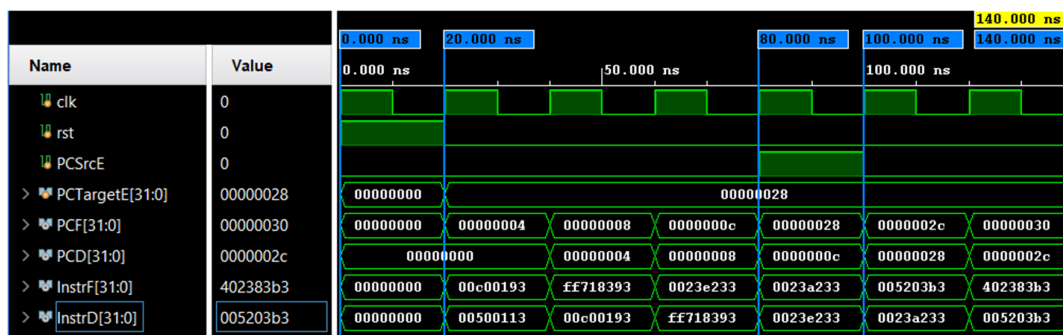
4.1. KẾT QUẢ MÔ PHỎNG DẠNG SÓNG CÁC KHỐI

4.1.1. Khởi lấy lệnh

Hình 4.1 mô tả Khối lấy lệnh bao gồm các thành phần chính: bộ đếm chương trình, bộ nhớ lệnh, bộ cộng, bộ chọn và thanh ghi. Khi hệ thống khởi động, tín hiệu rst đặt lại PC về giá trị ban đầu. Tín hiệu clk đồng bộ hóa các hoạt động. Bộ đếm chương trình xác định địa chỉ lệnh hiện tại và cập nhật địa chỉ tiếp theo thông qua bộ cộng và bộ chọn, trong khi bộ nhớ lệnh cung cấp lệnh tương ứng.



Hình 4.1: Sơ đồ RTL của Khối lấy lệnh



Hình 4.2: Dạng sóng của Khối lấy lệnh

Trong hình 4.2 mô tả chức năng của khối lấy lệnh được chia làm 4 trường hợp: Hình 4.2 mô tả chức năng của khối lấy lệnh trong 4 trường hợp:

Từ 0–20ns: Kiểm tra chức năng đặt lại hệ thống. Tín hiệu rst ở mức 1, các ngõ ra được đặt về mức 0.

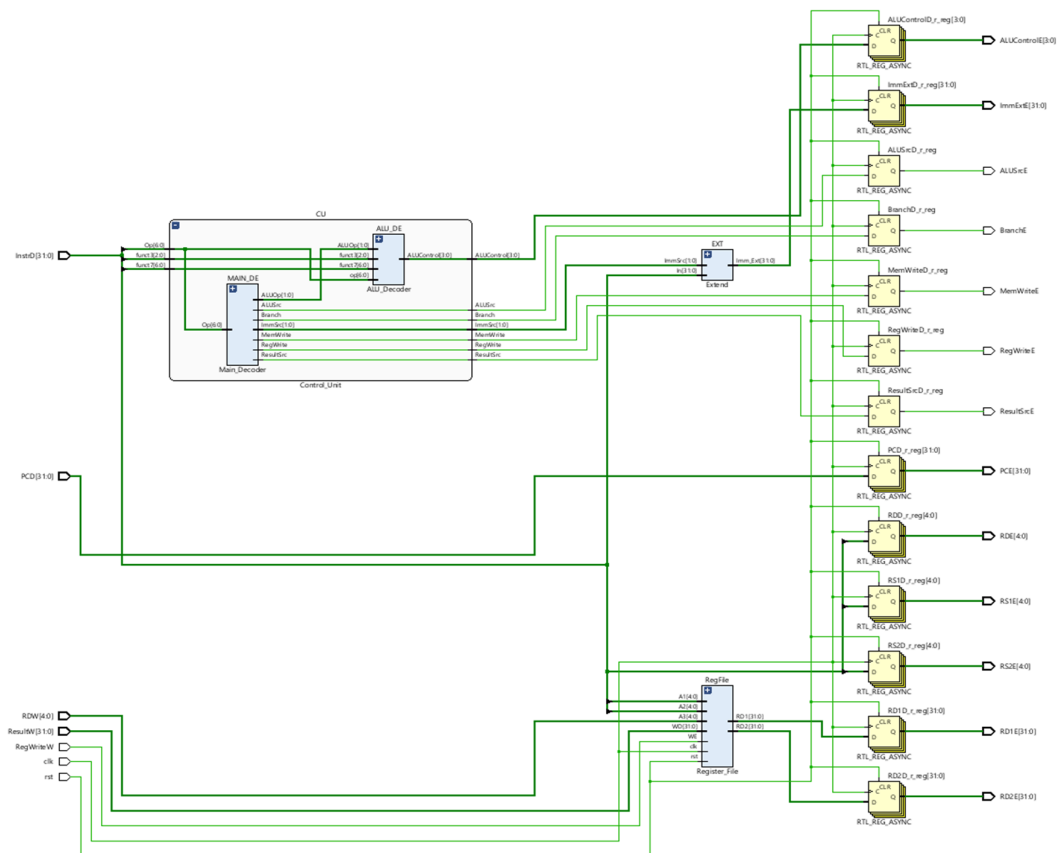
Từ 20–80ns: Kiểm tra chức năng tăng địa chỉ PC. Tín hiệu rst ở mức 0 và PCSrcE ở mức 0, giá trị PC tăng tuần tự (PCF = 32'h00000004, 32'h00000008,

32'h00000000c) với các lệnh tương ứng (InstrF = 32'h00c00193, 32'hff718393, 32'h0023e233). Ngõ ra PCD và InstrD chậm hơn 1 chu kỳ xung đồng hồ.

Từ 80–100ns: Kiểm tra chức năng nhảy địa chỉ. Tín hiệu rst ở mức 0 và PCSrcE ở mức 1, PC nhận giá trị đích (PCTargetE = 32'h00000028, PC = 32'h00000028), lệnh tương ứng là InstrF = 32'h23a233. Ngõ ra PCD và InstrD chậm hơn 1 chu kỳ xung đồng hồ.

Từ 100–140ns: Kiểm tra chức năng tăng địa chỉ PC sau khi kết thúc lệnh nhảy. Tín hiệu rst ở mức 0 và PCSrcE ở mức 0, PC tăng tuần tự (PCF = 32'h0000002c, 32'h00000030), lệnh tương ứng là InstrF = 32'h005203b3, 32'h402393b3). Ngõ ra PCD và InstrD chậm hơn 1 chu kỳ xung đồng hồ.

4.1.2. Khối giải mã lệnh



Hình 4.3: Sơ đồ RTL của Khối giải mã lệnh

Hình 4.3 mô tả sơ đồ RTL khối giải mã lệnh với các thành phần chính: Đơn vị điều khiển (CU), bộ giải mã chính, bộ giải mã ALU, tập hợp thanh ghi và các tín

hiệu điều khiển. Khi lệnh được tải vào CU, bộ giải mã chính phân tích lệnh và tạo ra các tín hiệu điều khiển, gửi đến bộ giải mã ALU và tập hợp thanh ghi. Bộ giải mã ALU xác định các phép toán cần thực hiện. Tín hiệu điều khiển từ CU điều phối các hoạt động như thực hiện phép toán trong ALU, đọc/ghi dữ liệu từ/đến bộ nhớ và cập nhật các thanh ghi. Tập hợp thanh ghi lưu trữ giá trị tạm thời và cung cấp dữ liệu cho các đơn vị khác khi cần.



Hình 4.4: Dạng sóng của Khối giải mã lệnh

Hình 4.4 mô tả chức năng của Khối giải mã lệnh được chia làm 8 trường hợp:

Từ 0 – 20ns: Kiểm tra chức năng đặt lại hệ thống. Tín hiệu rst ở mức 1, các ngõ ra về 0.

Từ 20 – 100ns: Kiểm tra chức năng ghi vào tập thanh ghi. Tín hiệu rst ở mức 0, lệnh InstrD = 32'hff718393 (addi x7, x3, -9). Từ 20 – 60ns, RegWriteW = 0 không cho phép ghi vào thanh ghi x3. Từ 60 – 100ns, RegWriteW = 1 cho phép ghi giá trị 32'habababab vào x3.

Từ 100 – 140ns: Kiểm tra lệnh loại tải. Tín hiệu rst ở mức thấp, lệnh InstrD = 32'h00512203 (lw x4 5(x2)). Các tín hiệu điều khiển: RegWriteE = 1, ResultSrcE = 1, MemWriteE = 0, BranchE = 0, ALUSrcE = 1, ALUControlE = 4'b0000, ImmExtE = 32'h00000005. Địa chỉ thanh ghi: RS1E = 5'h2, RDE = 5'h4. Giá trị RD1E = 0.

Từ 140 – 180ns: Kiểm tra lệnh loại S. Tín hiệu rst ở mức thấp, lệnh InstrD = 32'h005121a3 (sw x5 3(x2)). Các tín hiệu điều khiển: RegWriteE = 0, ResultSrcE = 0, MemWriteE = 1, BranchE = 0, ALUSrcE = 1, ALUControlE = 4'b0000, ImmExtE = 32'h00000003. Địa chỉ thanh ghi: RS1E = 5'h2, RS2E = 5'h5. Giá trị RD1E và RD2E = 0.

Từ 180 – 220ns: Kiểm tra lệnh loại B. Tín hiệu rst ở mức thấp, lệnh InstrD = 32'hfe208ce3 (beq x1, x2, -8). Các tín hiệu điều khiển: RegWriteE = 0, ResultSrcE = 0, MemWriteE = 0, BranchE = 1, ALUSrcE = 0, ALUControlE = 4'b0001, ImmExtE = 32'hffffffc. Địa chỉ thanh ghi: RS1E = 5'h1, RS2E = 5'h2. Giá trị RD1E và RD2E = 0.

Từ 220 – 260ns: Kiểm tra lệnh loại I. Tín hiệu rst ở mức thấp, lệnh InstrD = 32'h00208193 (addi x3, x1, 2). Các tín hiệu điều khiển: RegWriteE = 1, ResultSrcE = 0, MemWriteE = 0, BranchE = 0, ALUSrcE = 1, ALUControlE = 4'b0000, ImmExtE = 32'h00000002. Địa chỉ thanh ghi: RS1E = 5'h1, RDE = 5'h3. Giá trị RD1E = 0.

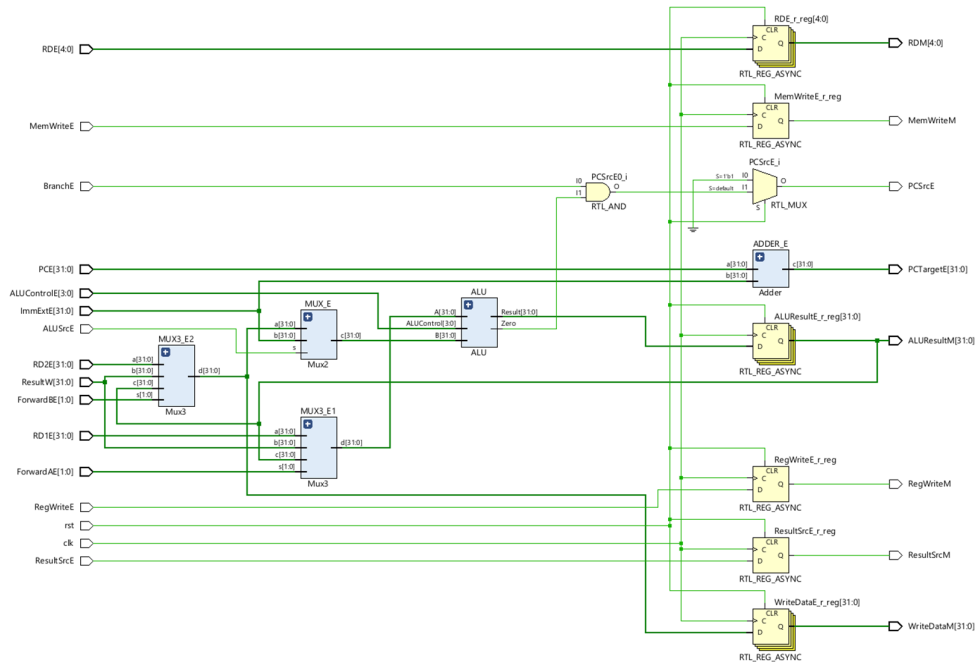
Từ 260 – 300ns: Kiểm tra lệnh loại R. Tín hiệu rst ở mức thấp, lệnh InstrD = 32'h002081b3 (add x3, x1, x2). Các tín hiệu điều khiển: RegWriteE = 1, ResultSrcE = 0, MemWriteE = 0, BranchE = 0, ALUSrcE = 0, ALUControlE = 4'b0000. Địa chỉ thanh ghi: RS1E = 5'h1, RS2E = 5'h2, RDE = 5'h3. Giá trị RD1E và RD2E = 0.

Từ 100 – 300ns: Kiểm tra chức năng ghi thanh ghi ID/EX. Tín hiệu PCD lần lượt xuất ra PCE từ 100 – 140ns (32'h00000004 = PCE), 140 – 180ns (32'h00000008 = PCE), 180 – 220ns (32'h0000000c = PCE), 220 – 260ns (32'h00000010 = PCE), 260 – 300ns (32'h00000014 = PCE).

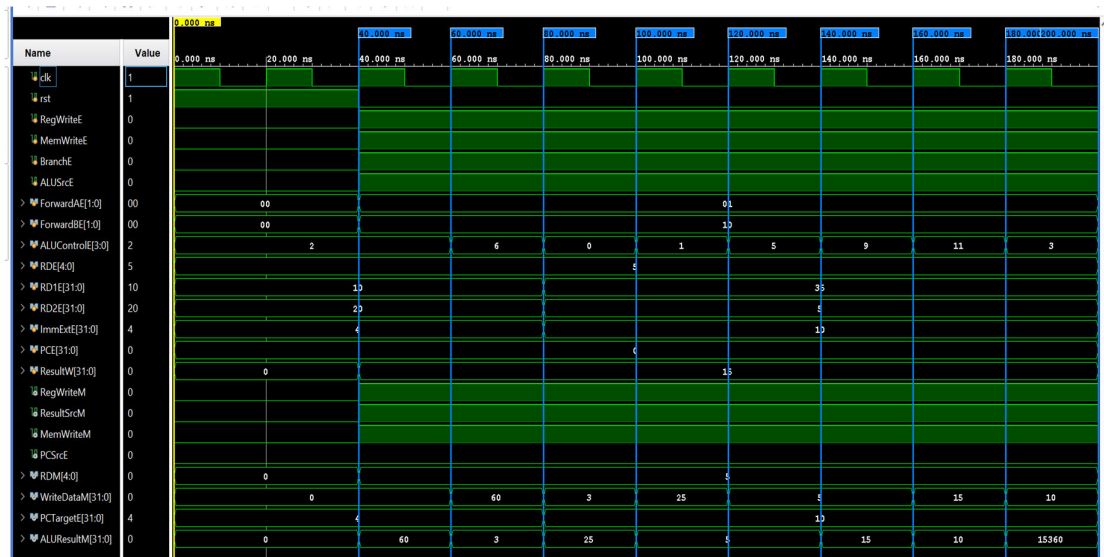
4.1.3. Khối thực thi

Hình 4.5 cho thấy sơ đồ RTL trên mô tả khối thực thi bao gồm các thành phần chính như bộ luận lý số học, bộ chọn ngõ vào, bộ chọn 2 ngõ vào và bộ cộng. Khối thực thi có nhiệm vụ thực hiện các phép tính số học, luận lý và tính toán địa chỉ bộ

nhớ, thực thi các lệnh và xử lý các ngoại lệ, quyết định đến hiệu suất chung của đường ống.



Hình 4.5: Sơ đồ RTL của Khối thực thi



Hình 4.6: Dạng sóng của Khối thực thi

Kết quả mô phỏng của khối thực thi qua Hình 4.6 như sau:

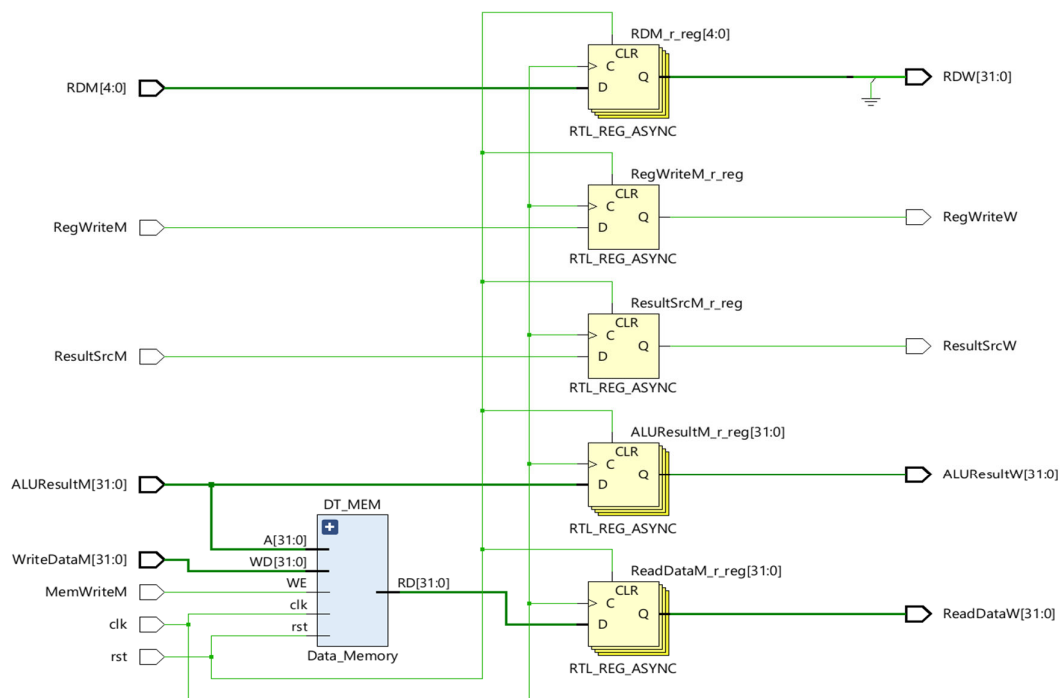
Từ 0 – 40ns: Kiểm tra chức năng đặt lại. Tín hiệu rst ở mức 1, các ngõ ra về 0.

Từ 40 – 200ns: Kiểm tra chức năng tính toán của ALU.

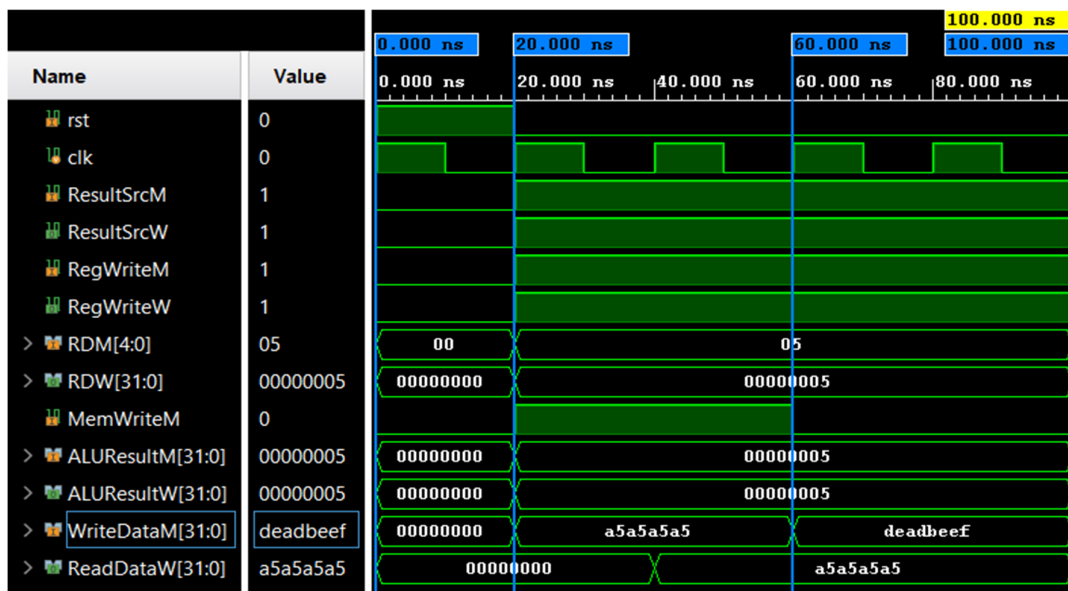
- 40 – 60ns: ALUControlE = 4'b0010 (phép nhân). ALU thực hiện phép nhân: $\text{SrcAE} * \text{SrcBE_out} = 15 * 4 = 60$, kết quả $\text{ALUResultE} = 60$, $\text{ZeroE} = 0$.
- 60 – 80ns: ALUControlE = 4'b0110 (phép chia). ALU thực hiện phép chia: $\text{SrcAE} / \text{SrcBE_out} = 15 / 4 = 3$, kết quả $\text{ALUResultE} = 3$, $\text{ZeroE} = 0$.
- 80 – 100ns: ALUControlE = 4'b0000 (phép cộng). ALU thực hiện phép cộng: $\text{SrcAE} + \text{SrcBE_out} = 15 + 10 = 25$, kết quả $\text{ALUResultE} = 25$, $\text{ZeroE} = 0$.
- 100 – 120ns: ALUControlE = 4'b0001 (phép trừ). ALU thực hiện phép trừ: $\text{SrcAE} - \text{SrcBE_out} = 15 - 10 = 5$, kết quả $\text{ALUResultE} = 5$, $\text{ZeroE} = 0$.
- 120 – 140ns: ALUControlE = 4'b0101 (phép XOR). ALU thực hiện phép XOR: $\text{SrcAE} \text{ XOR } \text{SrcBE_out} = 15 \text{ XOR } 10 = 5$, kết quả $\text{ALUResultE} = 5$, $\text{ZeroE} = 0$.
- 140 – 160ns: ALUControlE = 4'b1001 (phép OR). ALU thực hiện phép OR: $\text{SrcAE} \text{ OR } \text{SrcBE_out} = 15 \text{ OR } 10 = 15$, kết quả $\text{ALUResultE} = 15$, $\text{ZeroE} = 0$.
- 160 – 180ns: ALUControlE = 4'b1011 (phép AND). ALU thực hiện phép AND: $\text{SrcAE} \text{ AND } \text{SrcBE_out} = 15 \text{ AND } 10 = 10$, kết quả $\text{ALUResultE} = 10$, $\text{ZeroE} = 0$.
- 180 – 200ns: ALUControlE = 4'b0011 (phép dịch). ALU thực hiện phép dịch: $\text{SrcAE} \text{ dịch } \text{SrcBE_out} = 15 \text{ dịch } 10 = 15360$, kết quả $\text{ALUResultE} = 15360$, $\text{ZeroE} = 0$.

4.1.4. Khối truy cập bộ nhớ

Hình 4.7 cho thấy sơ đồ RTL trên mô tả khối truy cập bộ nhớ có bộ Data_Memory. Khối truy cập bộ nhớ thực hiện các hoạt động liên quan đến bộ nhớ. Cụ thể, khối này sẽ xử lý các lệnh tải và lưu trữ, truy xuất dữ liệu từ bộ nhớ hoặc lưu dữ liệu vào bộ nhớ, đảm bảo tính nhất quán của dữ liệu trong quá trình tương tác với bộ nhớ, ảnh hưởng lớn đến hiệu suất chung của đường ống.



Hình 4.7: Sơ đồ RTL của Khối truy cập bộ nhớ



Hình 4.8: Dạng sóng của Khối truy cập bộ nhớ

Kết quả mô phỏng khối truy cập bộ nhớ qua Hình 4.8 như sau:

Từ 0 – 20ns: Kiểm tra chức năng đặt lại. Tín hiệu rst ở mức 1, các ngõ ra về 0.

Từ 20 – 60ns: Kiểm tra chức năng đọc ghi dữ liệu bộ nhớ. Tín hiệu rst ở mức 0, MemWriteM = 1 cho phép ghi dữ liệu vào bộ nhớ tại địa chỉ ALUResultM =

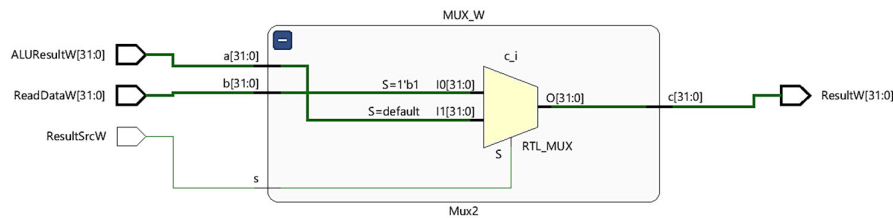
32'd5 với dữ liệu WriteDataM = 32'hA5A5A5A5. Kết quả ReadDataW = 32'ha5a5a5a5 sau 1 chu kỳ xung.

Từ 60 – 80ns: Kiểm tra chức năng không cho phép ghi dữ liệu vào bộ nhớ. Tín hiệu rst ở mức 0, MemWriteM = 0 không cho phép ghi dữ liệu, địa chỉ ALUResultM = 32'd5, dữ liệu WriteDataM = 32'hDEADBEEF. Kết quả ReadDataW = 32'ha5a5a5a5 cho thấy dữ liệu mới không được ghi.

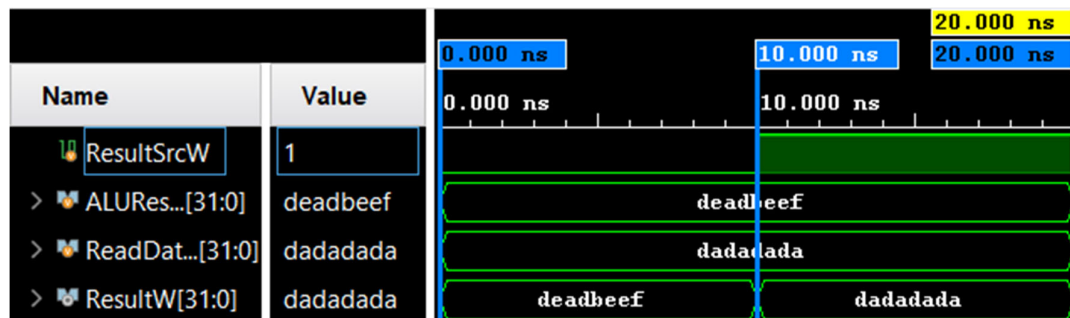
Từ 20 – 80ns: Kiểm tra các tín hiệu ResultSrcM, RegWriteM, RDM xuất ra qua thanh ghi MEM/WB với các giá trị: ResultSrcM = ResultSrcW = 1, RegWriteM = RegWriteW = 1, RDM = RDW = 5'h05.

4.1.5. Khối ghi lại

Khối ghi lại có trách nhiệm nhận kết quả từ các khối khác, ghi kết quả vào các thanh ghi đích, và cập nhật các thanh ghi trạng thái, hoàn tất chu kỳ thực thi lệnh.



Hình 4.9: Sơ đồ RTL của Khối ghi lại



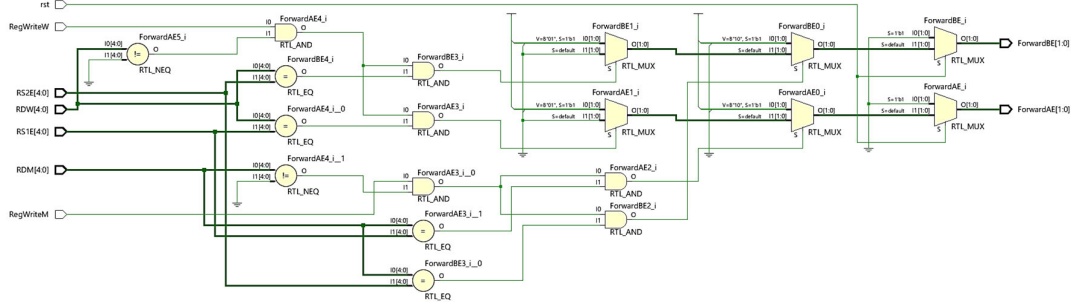
Hình 4.10: Dạng sóng của Khối ghi lại

Từ 0 – 10ns: Tín hiệu ResultSrcW = 0 chọn giá trị ALU làm đầu ra, ResultW = ALUResultW = 32'hdeadbeef.

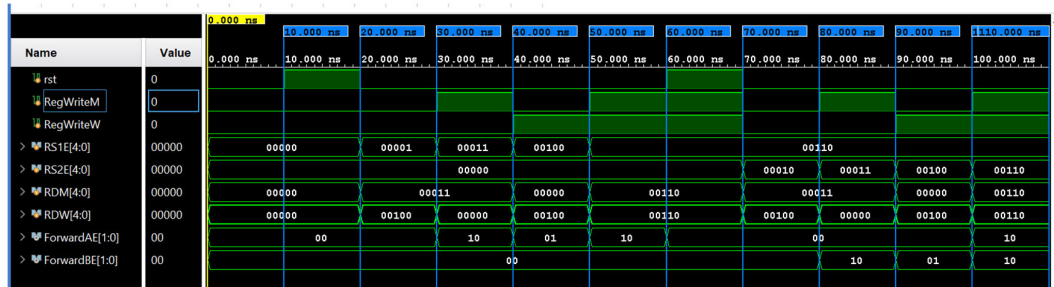
Từ 10 – 20ns: Tín hiệu ResultSrcW = 1 chọn giá trị bộ nhớ làm đầu ra, ResultW = ReadDataW = 32'hdadadada.

4.1.6. Khối xử lý xung đột

Khối xử lý xung đột có nhiệm vụ phát hiện và xử lý các tình huống xung đột trong đường ống như dữ liệu, điều khiển và cấu trúc thực hiện các chiến lược xử lý như chuyển tiếp để duy trì tính nhất quán của dữ liệu và tối ưu hóa hiệu suất của đường ống.



Hình 4.11: Sơ đồ RTL của Khối xử lý xung đột



Hình 4.12: Dạng sóng của Khối xử lý xung đột

Kết quả mô phỏng chức năng của Khối xử lý xung đột chia làm các trường hợp dưới:

Từ 0 – 10ns: Khởi tạo đầu vào, tất cả tín hiệu bằng 0.

Từ 10 – 20ns: Kiểm tra reset, rst = 1, các tín hiệu forwarding đặt về 2'b00.

Từ 20 – 30ns: Không có tín hiệu chuyển tiếp, rst = 0, RegWriteM và RegWriteW = 0, không có forwarding.

Từ 30 – 40ns: Chuyển tiếp từ giai đoạn truy cập bộ nhớ, rst = 0, RegWriteM = 1, RS1E = RDM, ForwardAE = 2'b10.

Từ 40 – 50ns: Chuyển tiếp từ giai đoạn ghi kết quả, rst = 0, RegWriteW = 1, RS1E = RDW, ForwardAE = 2'b01.

Từ 50 – 60ns: Chuyển tiếp từ cả hai giai đoạn, rst = 0, RegWriteM = 1, RegWriteW = 1, RS1E = RDM = RDW, ForwardAE = 2'b10.

Từ 60 – 70ns: Kiểm tra reset, rst = 1, các tín hiệu forwarding đặt về 2'b00.

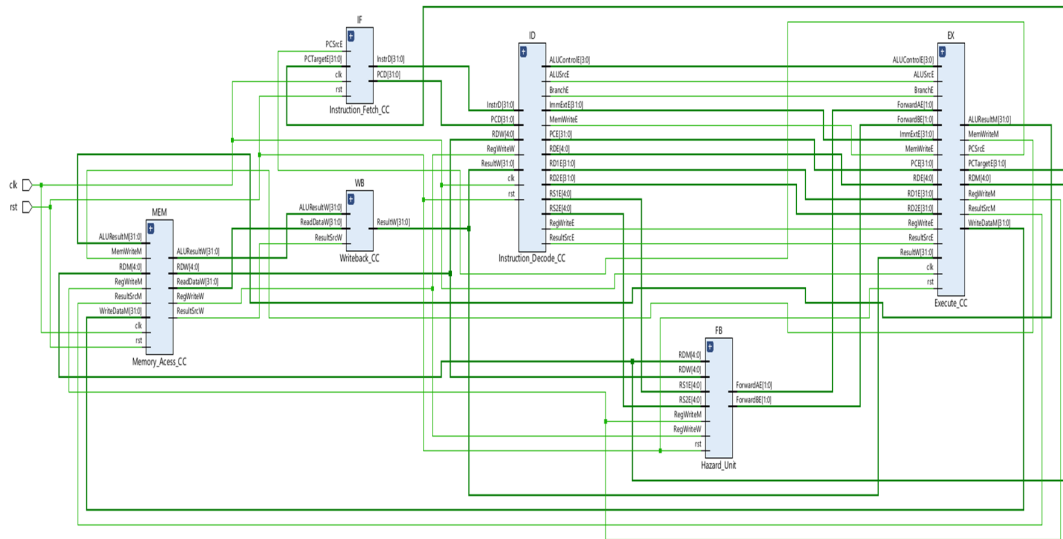
Từ 70 – 80ns: Không có tín hiệu chuyển tiếp, rst = 0, RegWriteM và RegWriteW = 0, không có forwarding.

Từ 80 – 90ns: Chuyển tiếp từ giai đoạn truy cập bộ nhớ, rst = 0, RegWriteM = 1, RS2E = RDM, ForwardBE = 2'b10.

Từ 90 – 100ns: Chuyển tiếp từ giai đoạn ghi kết quả, rst = 0, RegWriteW = 1, RS2E = RDW, ForwardBE = 2'b01.

Từ 100 – 110ns: Chuyển tiếp từ cả hai giai đoạn, rst = 0, RegWriteM = 1, RegWriteW = 1, RS2E = RDM = RDW, ForwardBE = 2'b10.

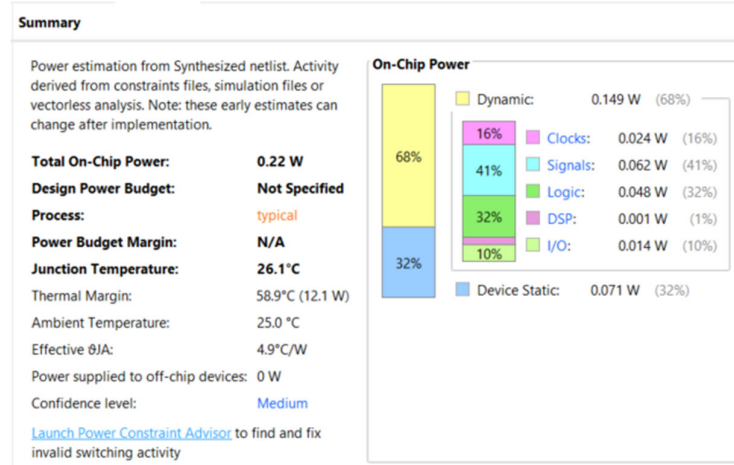
4.2. KẾT QUẢ MÔ PHỎNG TOÀN HỆ THỐNG



Hình 4.13: Sơ đồ RTL của Vi xử lý 32 bit

Sau khi thiết kế các khối thành phần, nhóm đã kết nối chúng để tạo sơ đồ RTL hoàn chỉnh của bộ vi xử lý 32 bit như Hình 4.13. Sơ đồ mô tả các khối chức năng chính và cách chúng tương tác trong quá trình thực thi lệnh. Lệnh được lấy từ bộ nhớ lệnh, giải mã để tạo ra các tín hiệu điều khiển, sau đó thực thi các phép toán số học hoặc logic. Nếu cần, dữ liệu sẽ được đọc hoặc ghi vào bộ nhớ. Kết quả được

ghi vào các thanh ghi qua khối ghi kết quả. Sơ đồ minh họa kỹ thuật đường ống, cho phép nhiều lệnh thực hiện đồng thời qua các giai đoạn khác nhau, sử dụng kiến trúc Harvard để tăng hiệu suất. Sơ đồ cũng hiển thị cách quản lý luồng dữ liệu và tín hiệu điều khiển giữa các khối để đảm bảo lệnh được thực thi chính xác và hiệu quả.



Hình 4.14: Công suất hoạt động của Bộ vi xử lý 32 bit

Hình 4.14 cung cấp một bản tóm tắt về ước tính tiêu thụ điện năng của bộ vi xử lý 32 bit. Tổng công suất tiêu thụ trên chip là 0.22 W, nhiệt độ điểm nổi là 26.1°C và biên nhiệt là 58.9°C (12.1 W). Nhiệt độ môi trường là 25.0°C với hiệu quả tản nhiệt là 4.9°C/W. Công suất tiêu thụ trên chip được chia thành hai phần chính: công suất động và công suất tĩnh. Công suất động là 0.149 W, chiếm 68% tổng công suất. Trong đó, các đồng hồ (Clocks) tiêu thụ 0.024 W (16%), các tín hiệu (Signals) tiêu thụ 0.062 W (41%), logic tiêu thụ 0.048 W (32%), DSP tiêu thụ 0.001 W (1%), và I/O tiêu thụ 0.014 W (10%). Công suất tĩnh của thiết bị là 0.071 W, chiếm 32% tổng công suất.

Kết quả mô phỏng dạng sóng ngõ ra của bộ vi xử lý 32 bit sẽ giúp chúng ta hiểu rõ hơn về cách các tín hiệu và dữ liệu di chuyển qua các giai đoạn khác nhau trong quá trình thực thi lệnh. Bằng cách kiểm tra này chúng ta có thể kiểm tra để đảm bảo rằng các lệnh được thực thi chính xác và hiệu quả, cũng như phát hiện và sửa

các lỗi có trong việc thiết kế. Kết quả mô phỏng được trình bày trong các Hình 4.15, Hình 4.16 và Hình 4.17, dựa vào Bảng 4.1, Bảng 4.2.

Bảng 4.1: Mã Assembly dùng mô phỏng chương trình

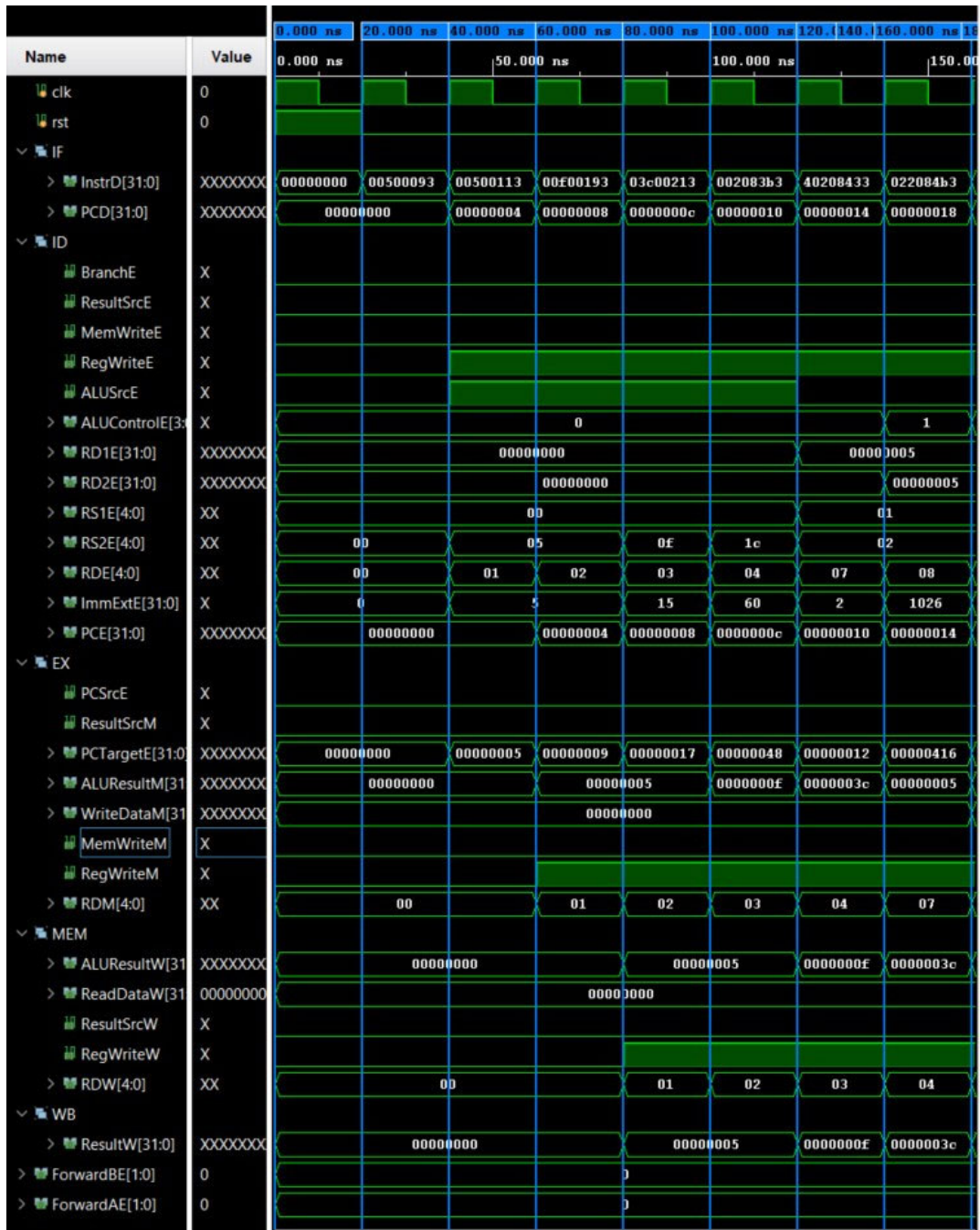
Nhãn	Mã Assembly	Mã HEX	Nhãn	Mã Assembly	Mã HEX
label:	addi x1, x0, 5	0x00500093	end:	lw x14, 4(x3)	0x0041A703
	addi x2, x0, 5	0x00500113		sw x4, 8(x3)	0x0041A423
	addi x3, x0, 15	0x00F00193		beq x1, x2, tag	0x00208663
	addi x4, x0, 60	0x03C00213		addi x5, x0, -15	0xFF100293
	add x7, x1, x2	0x002083B3	tag:	addi x6, x0, 255	0xFF00313
	sub x8, x1, x2	0x40208433		add x5, x3, x4	0x004182B3
	mul x9, x1, x2	0x022084B3		sub x7, x5, x3	0x403283B3
	div x10, x1, x2	0x0220C533		j end	0xFF1FF06F
	rem x11, x1, x2	0x0220E5B3			
	or x12, x1, x2	0x0020E633			
	sll x13, x1, x2	0x002096B3			

Bảng 4.2: Tóm tắt kết quả

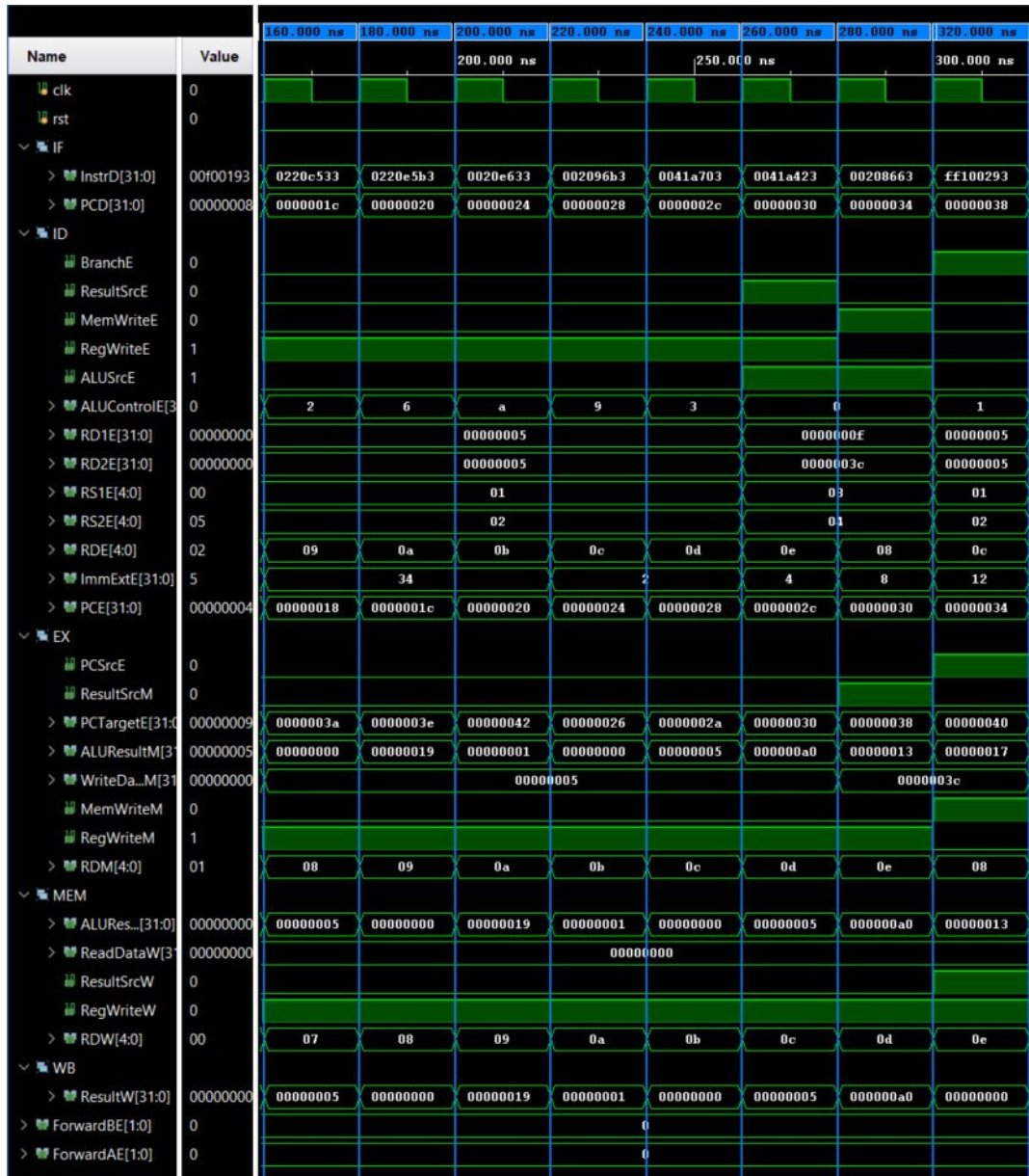
Trường hợp	Thời gian	Hoạt động
Đặt lại hệ thống	0 - 20ns	Tất cả các ngõ ra được đặt về mức 0
Lệnh loại I khởi tạo giá trị	20 - 120ns	Lệnh addi x1, x0, 5. Phép cộng : $ALUResultM = RD1E + ImmExtE = 32'd5$. $ResultW = 32'd5$; $RDW = 5'd1$.
	40 - 140ns	Lệnh addi x2, x0, 5. Phép cộng : $ALUResultM = RD1E + ImmExtE = 32'd5$. $ResultW = 32'd5$; $RDW = 5'd2$.
	60 - 160	Lệnh addi x3, x0, 15. Phép cộng : $ALUResultM = RD1E + ImmExtE = 32'd15$. $ResultW = 32'd15$; $RDW = 5'd3$.
	80 - 180ns	Lệnh addi x4, x0, 60. Phép cộng :

		$ALUResultM = RD1E + ImmExtE = 32'd60.$ $ResultW = 32'd60; RDW = 5'd4.$
Lệnh loại R	100-200ns	Lệnh add x7, x1, x2. Phép cộng : $x7 = x1 + x2 = 32'd5 + 32'd0 = 32'd5.$ $ResultW = 32'd5; RDW = 5'd7.$
	120-220ns	Lệnh sub x8, x1, x2. Phép trừ: $x8 = x1 - x2 = 32'd5 - 32'd5 = 32'd0.$ $ResultW = 32'd0; RDW = 5'd8.$
	140-240ns	Lệnh mul x9, x1, x2. Phép nhân: $x9 = x1 * x2 = 32'd5 * 32'd5 = 32'd25.$ $ResultW = 32'd25; RDW = 5'd9.$
	160-260ns	Lệnh div x10, x1, x2. Phép chia lấy nguyên: $nx10 = x1 / x2 = 32'd5 / 32'd5 = 32'd1.$ $ResultW = 32'd1; RDW = 5'd10.$
	180-280ns	Lệnh rem x11, x1, x2. Phép chia lấy dư: $x11 = x1 \% x2 = 32'd5 \% 32'd5 = 32'd0.$ $ResultW = 32'd0; RDW = 5'd11.$
	200-300ns	Lệnh or x12, x1, x2. Phép OR: $x12 = x1 x2 = 32'd5 32'd5 = 32'd5.$ $ResultW = 32'd5; RDW = 5'd12.$
	220-320ns	Lệnh sll x13, x1, x2. Phép dịch trái: $x13 = x1 << x2 = x1 << x2 = 32'd160.$ $ResultW = 32'd160; RDW = 5'd13.$
Lệnh tải	240-340ns	Lệnh lw x14, 4(x3). Tải giá trị vào thanh ghi: $ALUResultM = RD1E + ImmExtE = 32'd19.$ $ResultW = 32'd19; RDW = 5'd14.$
Lệnh lưu trữ	260-360ns	Lệnh sw x15, 8(x3). Lưu giá trị vào bộ nhớ: $ALUResultM = RD1E + ImmExtE = 32'd23.$ $ALUResultW = 32'd23; WriteDataM = 5'd60.$

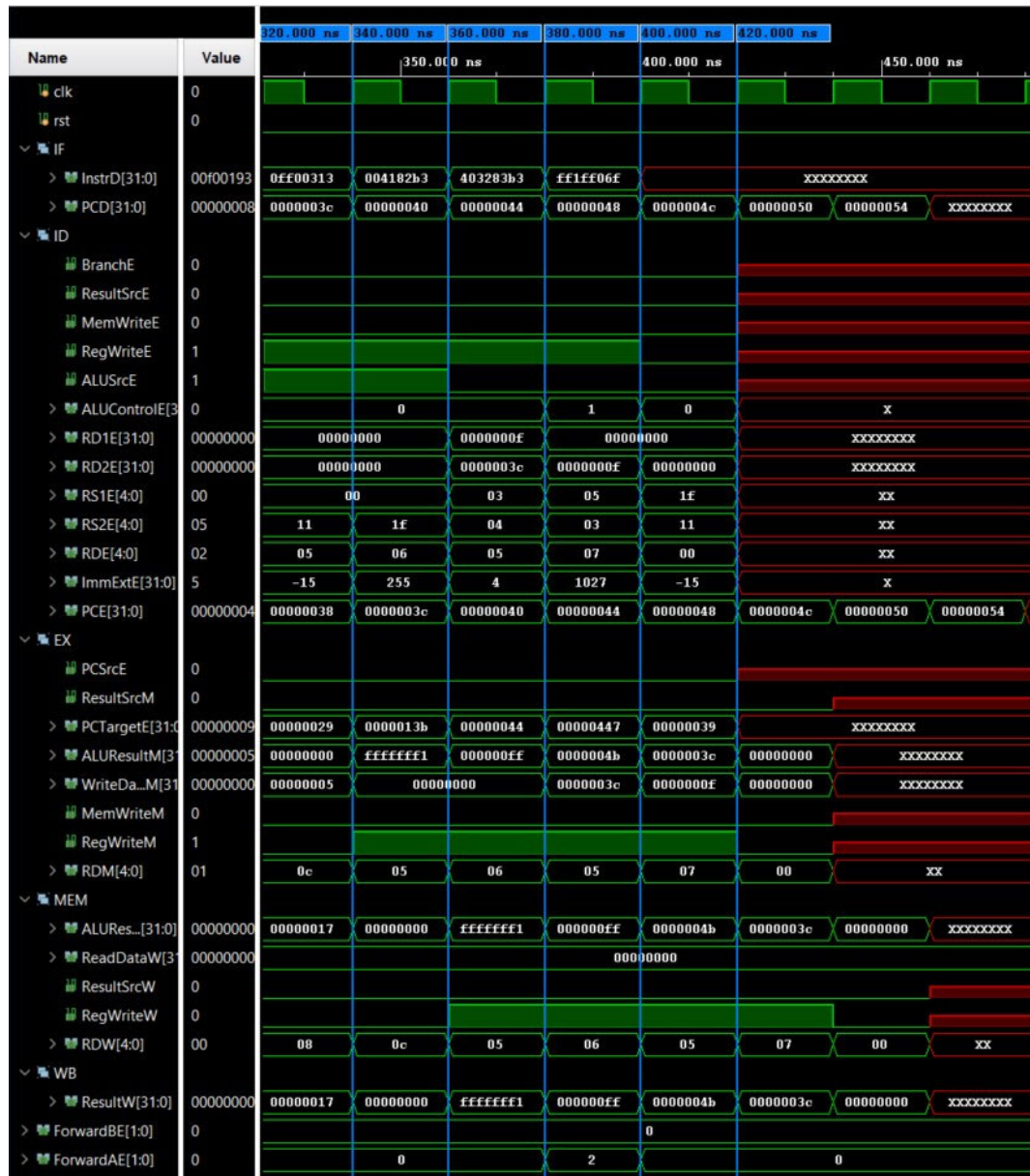
Lệnh nhảy	280-380ns	Lệnh beq x1, x2, tag. Lệnh nhảy có điều kiện: ALUResult = RD1E - RD2E = 32'd0. PCSrcE = 1, PCTargetE = 32'd40.
-----------	-----------	--



Hình 4.15: Dạng sóng của bộ vi xử lý 32 bit (từ 0 – 160ns)



Hình 4.16: Dạng sóng của bộ vi xử lý 32 bit (từ 160 – 320ns)



Hình 4.17: Dạng sóng của bộ vi xử lý 32 bit (từ 320 – 480ns)

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. KẾT LUẬN

Sau quá trình thực hiện đồ án thiết kế bộ vi xử lý 32 bit dựa trên kiến trúc tập lệnh RISC-V và kiến trúc Harvard, nhóm đã đạt được các yêu cầu sau:

- Thiết kế được một bộ vi xử lý 32 bit
- Mô phỏng được các chức năng của các khối còn trong bộ vi xử lý 32 bit.
- Mô phỏng được toàn bộ hệ thống vi xử lý 32 bit với một số lệnh cơ bản.

5.2. HƯỚNG PHÁT TRIỂN

Đề tài thực hiện “Thiết kế bộ vi xử lý 32 bit” được nhóm thực hiện đạt được các mục tiêu và yêu cầu thiết kế ban đầu đề ra nhưng còn hạn chế về khả năng và thời gian thực hiện nên kết quả thực hiện có thể thiếu sót một vài trường hợp chưa được tối ưu, nhóm đưa ra một số hướng phát triển như sau:

- Mô phỏng thêm nhiều lệnh hơn ở kiến trúc tập lệnh RISC-V.
- Thiết kế bộ vi xử lý với các phép tính số học có dấu chấm động.
- Nhúng lên kit FPGA để kiểm tra tín hiệu và chức năng thực tế.

TÀI LIỆU THAM KHẢO

- [1] – Hoàng Trung Hiếu, (2024), “*Những xu hướng công nghệ triển vọng năm 2024*”, *Báo Thế giới và Việt Nam*, <https://baoquocte.vn/nhung-xu-huong-cong-nghe-trien-vong-nam-2024-255740.html>.
- [2] – Tạp chí Thông tin & Truyền thông, (2023), “*Ngành công nghệ vi mạch và cơ hội phát triển ở Việt Nam*”, <https://ictvietnam.vn/nganh-cong-nghe-vi-mach-va-co-hoi-phat-trien-o-viet-nam-57472.html>.
- [3] – Báo chính phủ, (2023), “*Phát triển công nghiệp điện tử, vi mạch bán dẫn là đầu tư cho tương lai*” <https://baochinhphu.vn/phat-trien-cong-nghiep-dien-tu-vi-mach-ban-dan-la-dau-tu-cho-tuong-lai-102230906214454513.htm>.
- [4] – David A. Patterson, John L. Hennessy [2017], “*Computer Organization and Design RISC-V Edition: The Hardware Software Interface*”, Morgan Kaufmann 2017.
- [5] – Hyun Woo Oh, Kwon Neung Cho và Seung Eun Lee, (2020), “*Design of 32-bit Processor for Embedded Systems*”, 2020 International SoC Design Conference (ISOCC), IEEE, pp. 306–307.
- [6] – Mamun Bin Ibne Reaz, Md. Shabiul Islam và Mohd. S. Sulaiman, (2002), “*A single clock cycle MIPS RISC processor design using VHDL*”, ICONIP '02, Proceedings of the 9th International Conference on Neural Information Processing, Computational Intelligence for the E-Age (IEEE Cat. No.02EX575). IEEE, pp. 199–203.
- [7] – Safaa S. Omran và Hadeel S. Mahmood, (2013), “*Hardware Modelling of a 32-bit Single Cycle RISC Processor using VHDL*”, in ICIT 2013, The 6th International Conference on Information Technology.
- [8] – Poduel B. và cộng sự, (2015) “*Design and Implementation of Synthesizable 32-bit Four Stage Pipelined RISC Processor in FPGA Using Verilog/VHDL*,” Nepal journal of science and technology, Vol. 15, No.1, pp. 81–88.
- [9] – I. Thanga Dharsni, K. S. Pande and M. K. Panda, (2022), “*Optimized Hazard Free Pipelined Architecture Block for RV32I RISC-V Processor*” 2022 3rd International Conference on Smart Electronics and Communication (ICOSEC), Trichy, India.
- [10] – Synopsys, *What is RISC-V? – How Does it Work?*, <https://www.synopsys.com/glossary/what-is-risc-v.html>
- [11] – RISC-V, *History – RISC-V International* .<https://riscv.org/about/history/#origin>
- [12] – Sharma, S. (2024), *RISC-V Architecture: A Comprehensive Guide to the Open-Source ISA*, <https://www.wevolver.com/article/risc-v-architecture-a-comprehensive-guide-to-the-open-source-isa>

- [13] – Waterman, A. (2016), *Design of the RISC-V Instruction Set Architecture*. (Doctoral dissertation), EECS Department, University of California, Berkeley, 15 – 77.
- [14] – Nguyễn Văn Linh, (2019), *THIẾT KẾ BỘ NHỚ ĐỆM LỆNH CHO VI XỬ LÝ RISC-V*, Luận văn Thạc sĩ, Đại học Quốc gia TP.HCM, Trường Đại học Bách Khoa.
- [15] – Waterman, A. & Asanovic, K. (2017). *The RISC-V Instruction Set Manual*. SiFive Inc, tập 1, 27-102.
- [16] – Admin (2023). Kiến trúc RISC, CISC là gì? Sự khác biệt giữa RISC và CISC CPU. Yeuphancung. <https://yeuphancung.com/tim-hieu-kien-truc-risc-cisc/>
- [17] – Patterson, D. A., & Hennessy, J. L. (2018). *Computer Organization and Design: The Hardware/Software Interface: RISC-V Edition*. Waltham, MA: Morgan Kaufmann, 262–317.
- [18] – TS.Phạm Công Thắng, *Kỹ thuật đường ống và kiến trúc nâng cao*, Trường Đại học Bách Khoa Đại học Đà Nẵng, chương 11.
- [19] – Awati, R. (2022). *What is pipelining?*. TechTarget. <https://www.techtarget.com/whatis/definition/pipelining>
- [20] – Hennessy, J.L., Patterson, D.A. và Krste Asanović (2012). *Computer architecture : a quantitative approach*. Waltham, MA: Morgan Kaufmann.,C1 – C82
- [21] – GeeksforGeeks. (2016). *Computer Organization and Architecture | Pipelining | Set 2 (Dependencies and Data Hazard)*. <https://www.geeksforgeeks.org/computer-organization-and-architecture-pipelining-set-2-dependencies-and-data-hazard/>.
- [22] – Hoang Xuan Dau. (2010). *Bai giang kien truc may tinh*. Hoc vien buu chinh vien thong.